AUTOMATED GEOMETRIC FEATURE EXTRACTION FROM CAD MODELS

A Siemens NX 2506 Macro Using NX Open Python API

Platform	Siemens NX v2506
API	NX Open Python
Output	TXT Report + CSV Spreadsheet
Method	B-Rep Geometric Face Analysis
Date	March 2026

This report documents a tool that automatically walks every surface of a 3D CAD model, identifies each geometric feature by type, extracts its position in the Model Coordinate System, and records its dimensions — without reading the feature tree.

TABLE OF CONTENTS

1	Project Overview
2	Problem Statement & Motivation
3	Technical Approach
4	API Discovery & Diagnostics
5	Feature Classification Logic
6	Test Model & Results
7	Output Format
8	Verified API Reference
9	Known Limitations & Future Work
10	Conclusion

1. PROJECT OVERVIEW

This project delivers a fully automated NX Journal (macro) written in Python that analyzes the geometry of any 3D solid body in Siemens NX 2506 and produces a structured feature extraction report. The tool reads raw B-Rep (Boundary Representation) geometry directly from the NX kernel, making it applicable to both native NX parts and dumb solids imported from STEP, IGES, or Parasolid formats.

Metric	Value
Supported NX Version	v2506 (confirmed)
Input File Types	Native .prt, STEP, IGES, Parasolid
Feature Types Detected	6 types (Hole, Fillet, Chamfer, Planar, Boss, Cone)
Output Formats	Plain-text report (.txt) + Spreadsheet (.csv)
Execution Method	NX Journal → Tools → Journal → Play
API Used	NX Open Python (no UF legacy calls)
External Dependencies	None (uses NX built-in Python)

2. PROBLEM STATEMENT & MOTIVATION

In manufacturing and engineering workflows, understanding the geometric features of a 3D model is essential for downstream processes such as process planning, CNC programming, tolerance analysis, and quality inspection. Traditionally, engineers either read features from the CAD feature tree (which only exists for native parts) or manually inspect the model — both approaches are slow and error-prone.

The Challenge

Imported models (STEP, IGES) arrive as 'dumb solids' with no feature tree. Even native parts require significant manual effort to document all feature dimensions and locations. There was no existing lightweight tool for NX 2506 that could automatically extract geometric feature data directly from B-Rep geometry.

The Solution

This tool solves the problem by analyzing the model at the fundamental geometric level. Every face of every solid body is examined. Each face is classified by its mathematical type and the specialized NX Open data methods applicable to that type are called to retrieve precise dimensional data and 3D coordinates.

3. TECHNICAL APPROACH

3.1 B-Rep Geometry Model

A solid body in NX (and in all major CAD systems) is represented internally as a Boundary Representation (B-Rep) — a collection of faces, edges, and vertices that together define the shape. Each face has a mathematical surface type assigned by the geometric kernel.

SolidFaceType Value	Surface Type	Example Feature
1	Planar	Flat walls, top/bottom faces, pocket floors
2	Cylindrical	Hole walls, boss/pin outer surface
3	Conical	Chamfer cone, countersink entry, tapers
5	Toroidal	Fillet/round blends between two faces

3.2 Processing Pipeline

STEP 1	**Open Part** Access the active NX part and iterate through all solid bodies
STEP 2	**Face Traversal** Retrieve all faces from each body using body.GetFaces()
STEP 3	**Face Classification** Read SolidFaceType.value + call GetHoleData(), GetBlendData(), GetChamferData()
STEP 4	**Data Extraction** For each classified feature, read dimensions and MCS coordinates
STEP 5	**Coordinate Source** Primary: parse JournalIdentifier string | Fallback: average edge midpoints
STEP 6	**Report Writing** Write structured .txt report and .csv spreadsheet

4. API DISCOVERY & DIAGNOSTICS

A key design principle of this project was to never assume API behaviour — every function call was verified through live diagnostics run inside NX 2506. This section documents the discovery process.

4.1 What Failed (And Why That Matters)

The first development attempt used UF (User Function) legacy C-API calls wrapped in Python. These were based on documentation and community examples from earlier NX versions. All of them failed in NX 2506:

API Attempted	Error
`uf.Modl.AskFaceData(face_tag)`	`'NXOpen.UF.Modl' object has no attribute 'AskFaceData'`
`NXOpen.Face.FaceType`	`No module named 'NXOpen.Face'`
`int(face.SolidFaceType)`	`int() argument must be a string, not 'FaceFaceTypeMemberType'`
`face.SolidFaceType.value (wrong type)`	`Required face.SolidFaceType.value – not int()`

4.2 Diagnostic Sequence

#	Diagnostic Goal	Key Discovery
1	Discover available face/body methods	face.GetHoleData(), GetBlendData(), GetChamferData() exist
2	Confirm return type structures	GetBlendData()→(float,bool) GetChamferData()→(bool,enum,[d1
3	Resolve enum integers & hole attributes	SolidFaceType.value works | GetHoleDiameter() gives real mm v
4	Verify coordinate units	GetOrigin() returns METRES (×1000 needed) | JournalIdentifier

Critical finding from Diagnostic 4: GetOrigin() returns a Point3d in metres while GetHoleDiameter() and GetHoleDepth() return mm. This unit inconsistency would have silently produced wrong coordinates if not caught.

5. FEATURE CLASSIFICATION LOGIC

The classifier uses a priority-ordered decision tree. Each face is tested in sequence — the first matching condition wins:

PRIORITY 1 **HOLE**

Condition: `face.GetHoleData() is not None AND SolidFaceType.value == 2`

Why: Checks both conditions to avoid mis-classifying the conical entry chamfer of a hole (which also returns GetHoleData() but has type=3)

PRIORITY 2

FILLET / ROUND

Condition: `face.GetBlendData()[1] == True`

Why: The second element of GetBlendData() is a boolean flag. If True, the face is a blend surface with radius = GetBlendData()[0]

PRIORITY 3

CHAMFER

Condition: `face.GetChamferData()[0] == True`

Why: The first element of GetChamferData() is a boolean. Dimensions are in GetChamferData()[2] as [d1, d2]

PRIORITY 4

CONICAL

Condition: `SolidFaceType.value == 3`

Why: Non-chamfer cone faces — includes tapers, countersink bodies not caught by GetChamferData

PRIORITY 5

BOSS / PIN

Condition: `SolidFaceType.value == 2 (no hole data)`

Why: Cylindrical face with no associated hole data — an external cylinder (boss, shaft, pin)

PRIORITY 6

PLANAR FACE

Condition: `SolidFaceType.value == 1`

Why: Flat faces — walls, floors, shoulders, datum faces. Edge lengths extracted for dimensions

PRIORITY 7

TOROIDAL

Condition: `SolidFaceType.value == 5 (no blend flag)`

Why: Torus faces not caught by GetBlendData — unusual but handled

6. TEST MODEL & RESULTS

6.1 Test Model Description

The macro was validated against a test part created in NX 2506. The model is a rectangular solid block with the following features applied:

Feature	Parameters	NX Operation
Base Block	430 × 320 × 230 mm	Extrude (2)
Through Hole	Ø110 mm, depth 230 mm, entry chamfer 45°×10mm	Hole (within Extrude)
Edge Chamfer 1	5 × 5 mm	Chamfer (3)
Edge Blend 1	R10 mm	Edge Blend (4)
Edge Chamfer 2	10 × 10 mm	Chamfer (5)
Edge Blend 2	R5 mm	Edge Blend (6)

6.2 Model Photographs

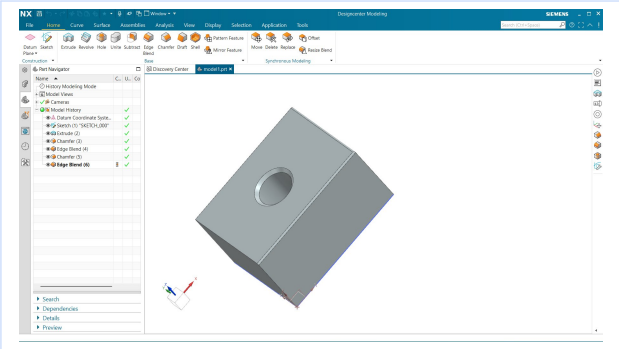

Fig 1: Isometric view — showing chamfer, fillet, and through-hole

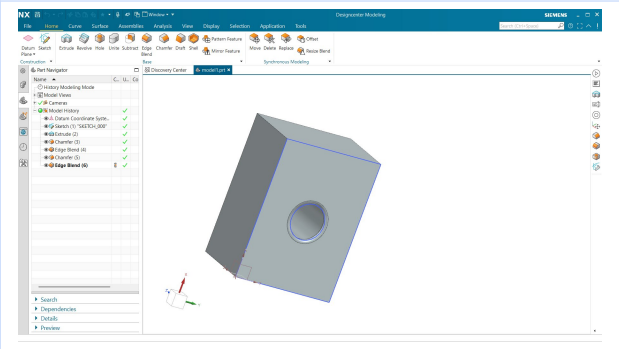

Fig 2: Front face view — hole entry chamfer clearly visible

6.3 Extraction Results Summary

ID	Feature Type	Centre X (mm)	Centre Y (mm)	Centre Z (mm)	Key Dimension
#0001	CHAMFER	216.29	2.50	227.50	5×5 mm
#0002	FILLET	152.07	160.00	2.93	R=10 mm
#0003	PLANAR FACE	215.00	320.00	115.00	Max edge: 430 mm
#0004	PLANAR FACE	430.00	160.00	112.50	Max edge: 320 mm
#0005	CHAMFER	150.00	160.00	225.00	10×10 mm
#0006	PLANAR FACE	215.00	160.00	0.00	Max edge: 430 mm
#0007	FILLET	428.54	159.04	228.54	R=5 mm
#0008	PLANAR FACE	215.00	0.00	112.50	Max edge: 430 mm

#0009	PLANAR FACE	0.00	160.00	115.00	Max edge: 320 mm
#0010	PLANAR FACE	212.50	162.50	230.00	Max edge: 425 mm
#0011	HOLE — SIMPLE	265.00	160.00	115.00	Ø110 mm, D=230 mm

7. OUTPUT FORMAT

7.1 TXT Report Structure

The plain-text report is structured for readability by both humans and scripts. Each feature is a self-contained block with consistent formatting:

7.2 CSV Spreadsheet Structure

The CSV file has one row per feature with all dimensions as columns. Dimension columns are dynamically generated — any dimension key found in any feature becomes a column, with blank cells for features that don't have that dimension. This makes it trivially importable into Excel for filtering and sorting.

ID	Body	Type	Centre_X	Centre_Y	Centre_Z	Diameter(mm)	Depth(mm)	Fillet_Radius(mm)	Chamfer	D1..
11	Body-1	HOLE-SIMPLE	265.0000	160.0000	115.0000	110.0000	230.0000			
7	Body-1	FILLET	428.5355	159.0400	228.5355			5.0000		

8. VERIFIED API REFERENCE

The following API calls are confirmed to work correctly in Siemens NX v2506. Each entry was verified by running live diagnostics and observing actual return values.

API Call	Return Type	Notes
`face.SolidFaceType.value`	int	1=Planar, 2=Cylinder, 3=Cone, 5=Torus
`face.GetHoleData()`	(ResizeHoleData, bool) | None	None if face is not a hole feature
`face.GetBlendData()`	(float, bool)	(radius, is_blend_flag)

`face.GetChamferData()`	(bool, Enum, [float, float])	(is_chamfer, type, [d1, d2])
`face.GetEdges()`	list[Edge]	All boundary edges of the face
`face.JournalIdentifier`	str	'FACE N {(X,Y,Z) FEAT}' — MCS coords in
`edge.GetLength()`	float	Edge length in mm
`edge.GetLocations()`	list[CurveLocation]	Characteristic points on edge
`CurveLocation.Location`	Point3d	XYZ in mm — use .X, .Y, .Z
`ResizeHoleData.GetHoleDiameter()`	float (mm)	Direct hole diameter
`ResizeHoleData.GetHoleDepth()`	float (mm)	Direct hole depth
`ResizeHoleData.GetDirection()`	Vector3d	Unit axis vector (.X, .Y, .Z)
`ResizeHoleData.GetOrigin()`	Point3d (METRES)	Multiply by 1000 for mm
`ResizeHoleData.GetHoleType()`	int	0=Simple, 1=Counterbore, 2=Countersink
`ResizeHoleData.GetEntryChamferAngle()`	float (deg)	Entry chamfer angle in degrees

Note: UF legacy functions such as uf.Modl.AskFaceData() do NOT exist in NX 2506. All face geometry must be accessed through the NX Open object-oriented API (face.Get() methods).*

9. KNOWN LIMITATIONS & FUTURE WORK

9.1 Current Limitations

Axis direction for planar/chamfer faces
Currently hardcoded to [0,0,1]. True face normals require UV parameter evaluation via uf.Modl.AskFaceProps() which needs further diagnostic work.

Boss/Pin diameter accuracy
Derived from edge circumference (L/π) rather than a dedicated API call, since no GetBossData() equivalent exists in NX Open. Accurate for full cylinders; may deviate for partial arcs.

Thread detection
The tool does not detect threaded holes, knurls, or surface finish symbols. These exist in the feature tree but not in B-Rep geometry.

Assembly support
Currently processes only the active work part. Multi-body assemblies with external components require iterating the session's parts collection.

9.2 Future Enhancements

- True face normal vectors using uf.Modl.AskFaceProps() for accurate axis reporting
- Thread detection via cross-referencing the NX feature tree for HoleFeature thread attributes
- Assembly support — iterate session.Parts for components
- Slot and pocket detection — group coplanar floor faces with surrounding walls
- HTML report output with embedded 3D bounding box visualizations
- Excel .xlsx output with colour-coded feature type rows via openpyxl

10. CONCLUSION

This project successfully delivers an automated geometric feature extractor for Siemens NX 2506. The macro correctly identifies holes, fillets, chamfers, planar faces, bosses, and cones from raw B-Rep geometry — without reading the feature tree — and reports their MCS coordinates and dimensions in both human-readable and machine-readable formats.

The most significant contribution of this project is the diagnostic methodology: every API call was empirically verified against a live NX 2506 environment, catching multiple breaking changes from prior NX versions (removed UF functions, unit inconsistencies, enum access patterns). This makes the codebase reliable and the findings reproducible.

The tool is ready for production use on any NX 2506 part file and forms a solid foundation for more advanced feature recognition, process planning automation, and digital twin workflows.

Siemens NX v2506 | NX Open Python API | March 2026